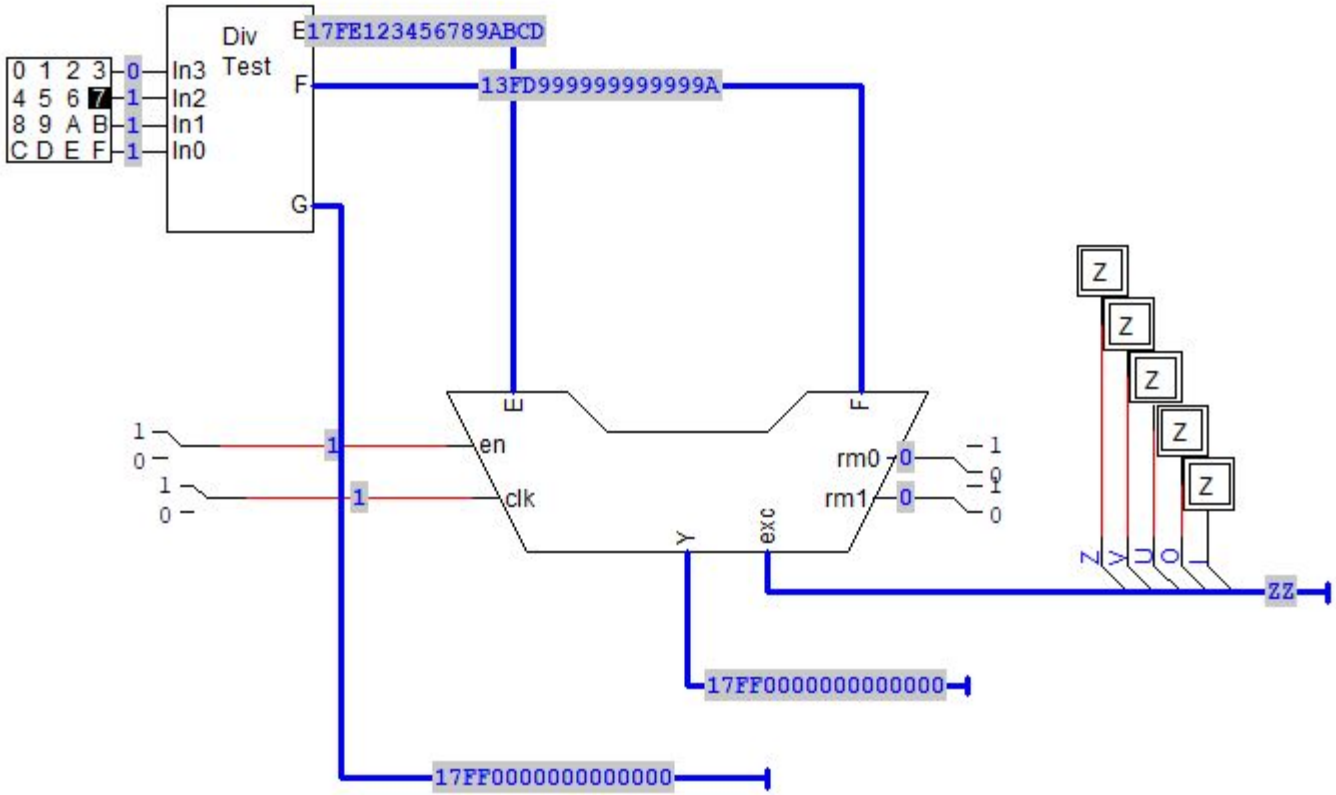
Divider Exponent

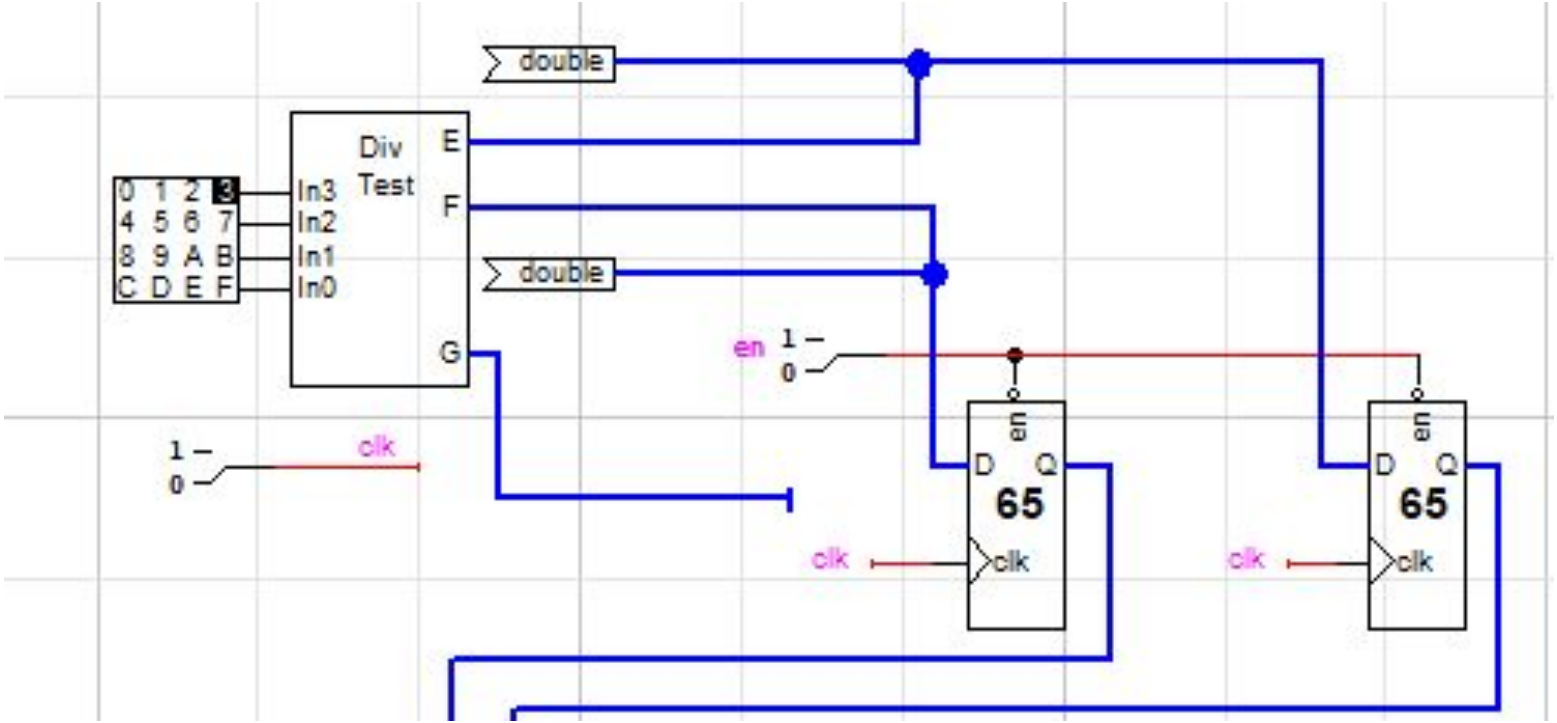
Nahom Solomon
Jaskaran Mann


0 1 2 3
4 5 6 7
8 9 A B
C D E F
0
1
1
1
In3
In2
In1
In0
Div
Test
E
F
G
17FE123456789ABCD
13FD999999999999A
1
0
1
0
1
1
en
clk
E
F
rm0
rm1
0
0
1
0
1
0
Y
exc
Z
Z
Z
Z
Z
ZZ
17FF0000000000000
17FF0000000000000

Cases

- Case 0
  - num: 3.0
  - denom: 1.375
  - results: 2.1818181818181817
- Case 1
  - num: 1.0
  - denom: 0.0
  - results: inf
- Case 2
  - num: 1.9999999999999998
  - denom: 1.0000000000000002
  - results: 1.9999999999999993
- Case 3
  - num: inf
  - denom: 4.9406564584124654e-324
  - results: inf
- Case 4
  - num: 0.0
  - denom: 0.0
  - results: 0,0xfff80000 # nan
- Case 5
  - num: 100.0
  - denom: 10.0
  - results: 10.0
- Case 6
  - num: 9.6276454555959567e+307
  - denom: 0.59999999999999998
  - results: 1.6046075759326596e+308
- Case 7
  - num: 9.6276454555959567e+307
  - denom: 0.40000000000000002
  - results: inf
- Case 8
  - num: 1.2345e-307
  - denom: 2.0
  - results: 6.1725e-308
- Case 9
  - num: 1.2345e-307
  - denom: 10.0
  - results: 1.2345000000000002e-308
- Case A
  - num: 4.9999999999999995e-308
  - denom: 10000000000000000
  - results: 4.9406564584124654e-324
- Case B
  - num: -1000000
  - denom: 3.0
  - results: -333333.33333333331
- Case C
  - num: -9.9999999999999995e-07
  - denom: 3.0
  - results: -3.3333333333333333e-07
- Case D
  - num: 0.4
  - denom: -0.1
  - results: -4
- Case E
  - num: -27.0
  - denom: -9.0
  - results: 3.0
- Case F
  - num: inf
  - denom: 3.0
  - results: inf

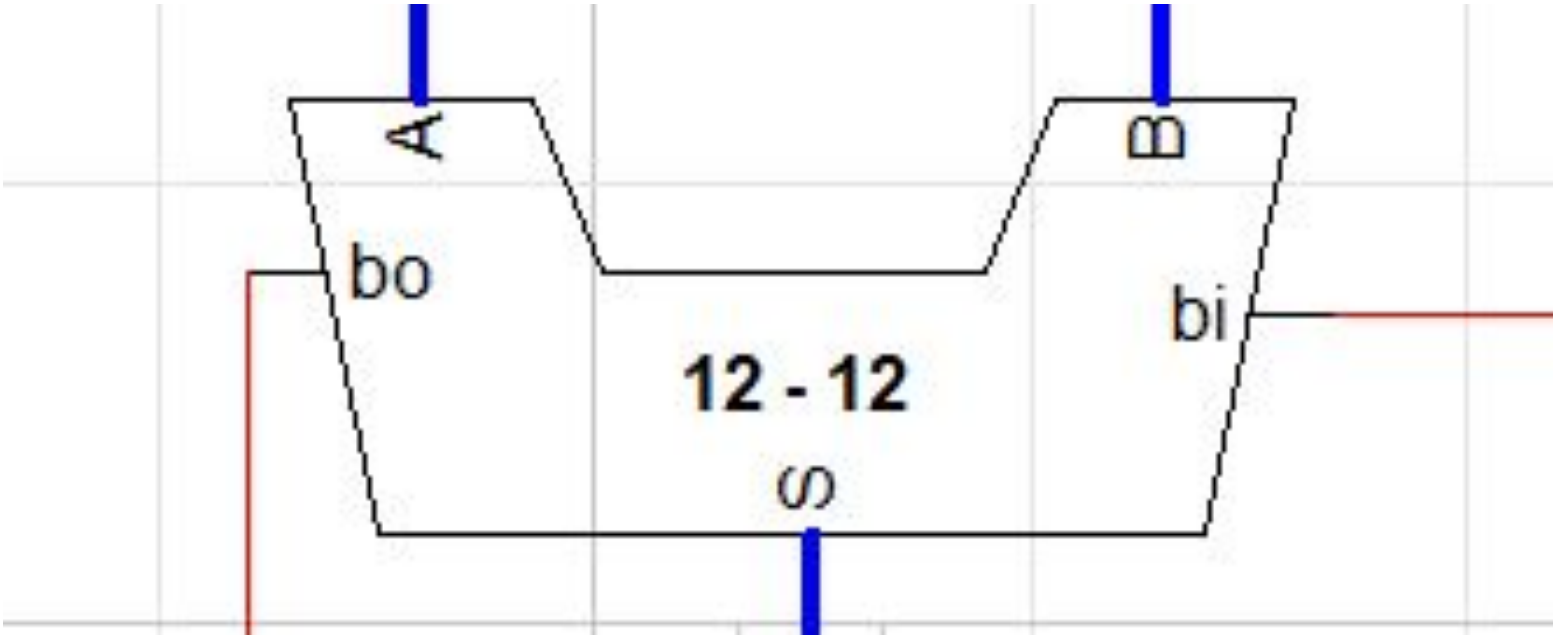
Stage 1: Input Loading



Stage 1: Input Loading

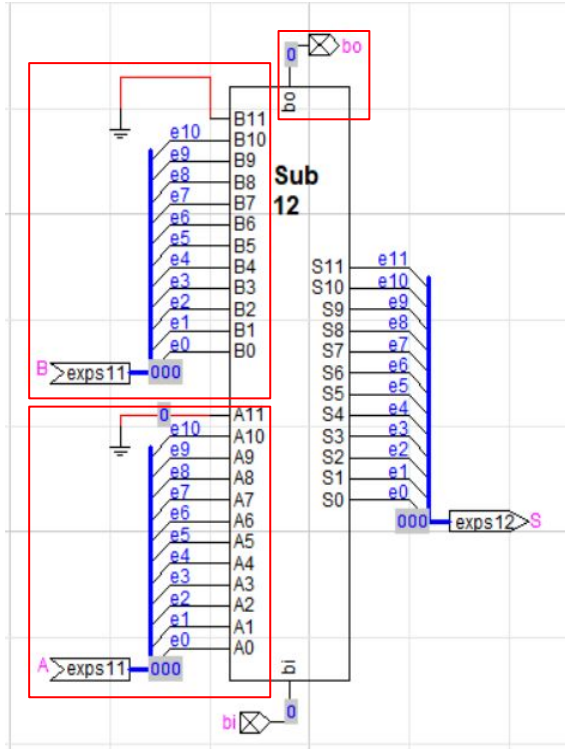
- The Divider Test allows us to select up to 16 test inputs with expected results to validate our circuits is performing the correct operation
- Inputs are display in the order of
  - 1 Hidden bit (m52)
  - 1 sign bit (sign)
  - 11 exponent bits (e10..e0)
  - 52 mantissa bits (m51..m0)
  - Total number of bits = 65 bits
- The inputs represent 64- bit floating- point numbers E (numerator), F (denominator), and G (expected results)
- E and F go into two 65-bit registers to capture and hold the values so they don't change during computation. They prepare the fields for the exponent processing steps.

Stage 2a: Exponent Subtraction



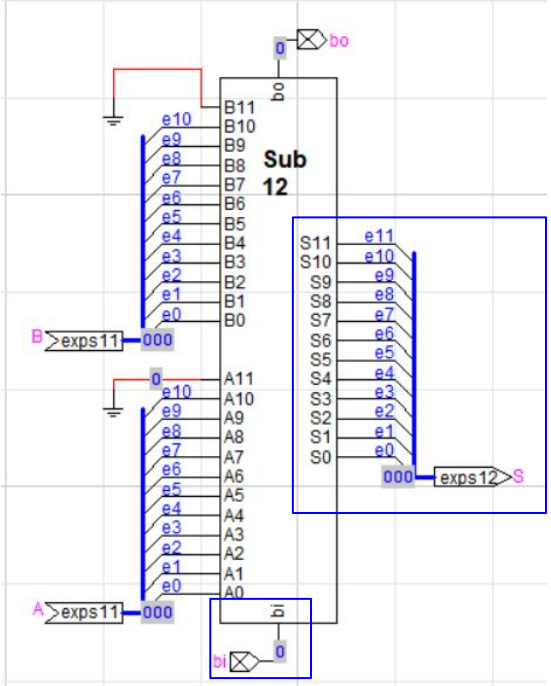
Stage 2a: Exponent Subtraction

- The exponent from B is subtracted from A. This is needed for floating-point division to compute the new exponent value
  - Result Exponent = A_exp - B_exp
- Inputs are A[11:0], B[11:0], and bi
  - A[11] and B[11] = 0
    - Grounded to force sign bit to be a positive input
  - A[10:0] and B[10:0] = 11-bit exponent
    - e10..e0
  - bi (bit in) controls what operation is being performed, which comes from the inverted value of the most significant mantissa bit (m55)
    - bi = 1
      - Performs two's complement subtraction (A - B - 1)
        - Two's complement: [A+(~B+1)]
    - bi = 0
      - Performs basic subtraction (A - B)

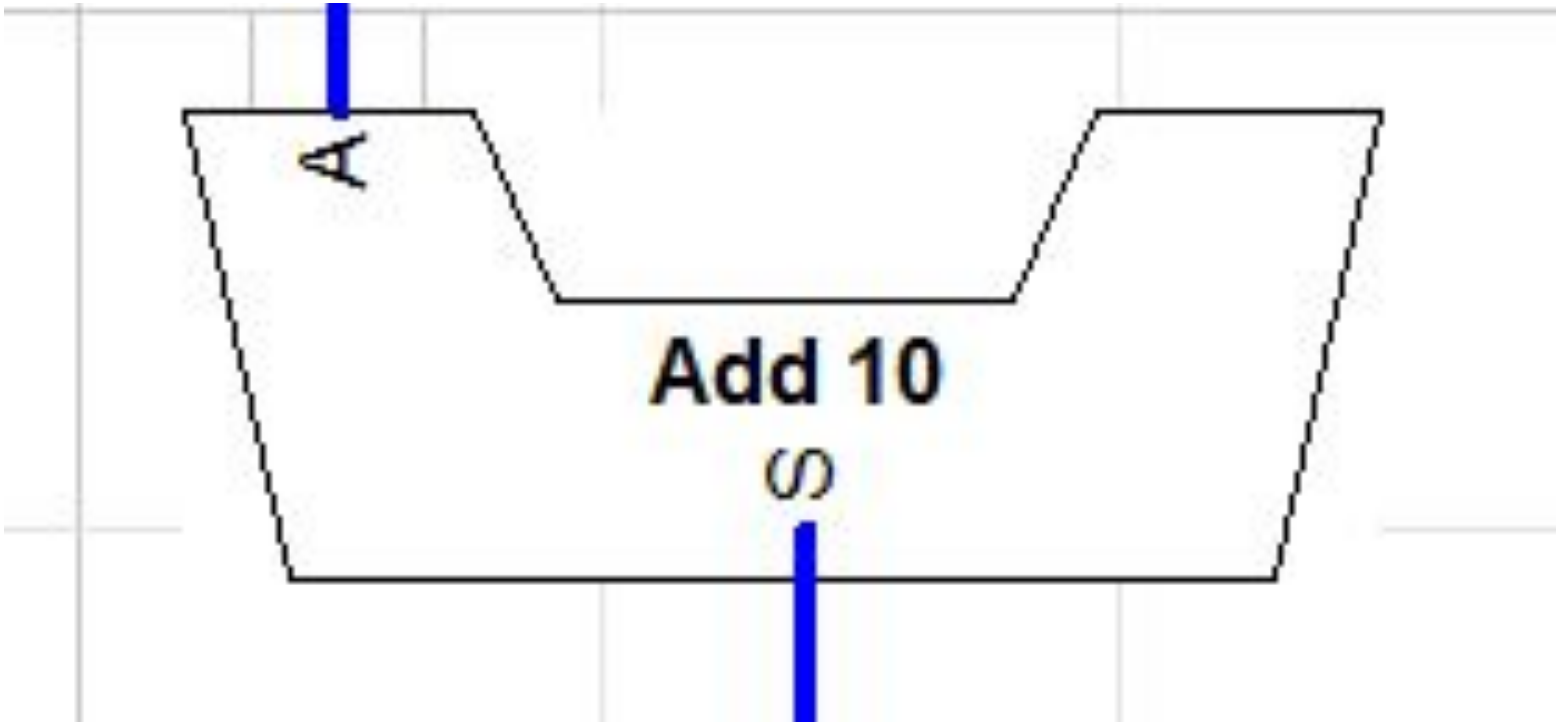


Stage 2a: Exponent Subtraction

- Outputs are S and bo
  - S (sum) represents the 12-bit signed result
    - The exponent difference
      - Which can be positive or negative, in two's complement form
    - e11..e0
  - bo (bit out) is the borrow-out flag, which provides comparison feedback without modifying the result
    - bo = 1
      - Borrow happened (A < B)
    - bo = 0
      - no Borrow happened (A >= B)

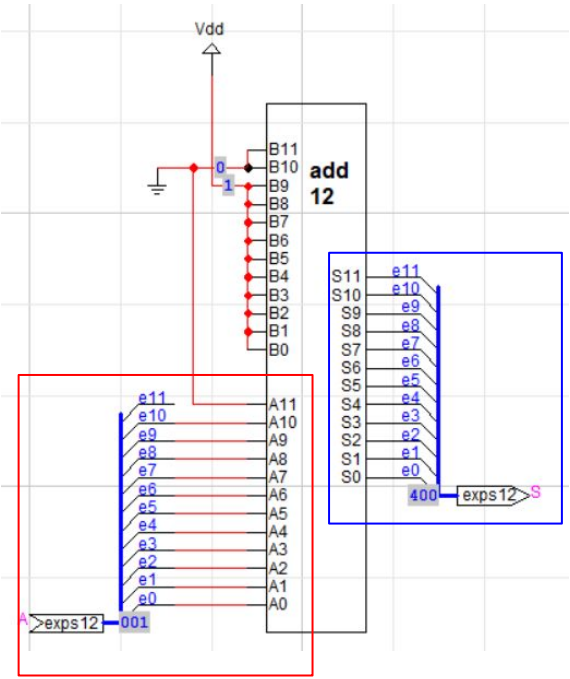


Stage 2b: Bias Addition

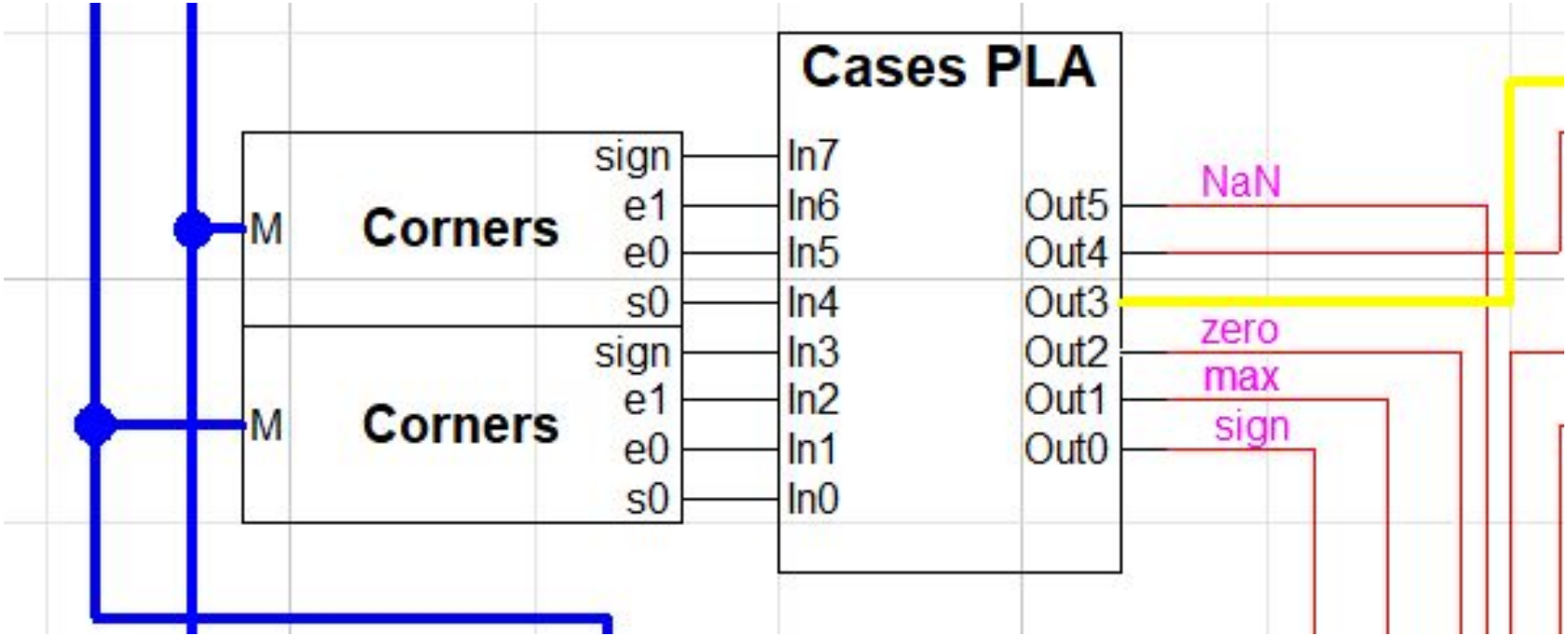


Stage 2b: Bias Addition

- After subtracting the exponents in Stage 2a, the result is unbiased
  - Means it does not match the IEEE-754 floating- point format, so we need to add 1023 (decimal)
    - Result Exponent (biased) = Result Exponent (unbiased) + 1023 (decimal)
  - Inputs are A[11:0], the result exponent (unbiased) from the subtractor
    - e11..e0
  - To add 1023 (decimal), we hardcode
    - B[11:10] = 0
    - B[9:0] = 1
  - Output is S, which is the 12- bit exponent (biased)



Stage 3: Exception Detection (PLA)



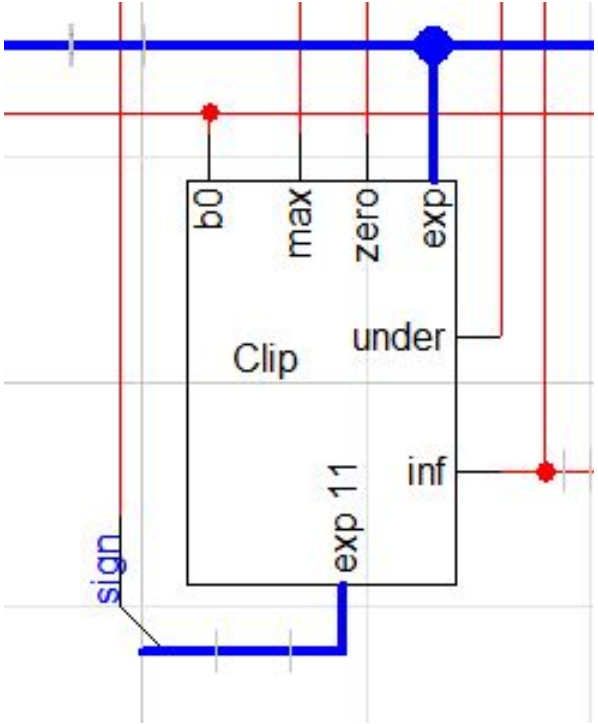
Stage 3: Exception Detection (PLA)

- We detect special floating-point corner case be examining the signs, exponents, and significands of the inputs A and B
- Which generates four key signals with will be the inputs into the PLA
  - sign
    - identifies the sign bit
  - e1
    - Identifies if the exponents field is all 1’s
      - Special cases: Infinity or NaN
  - e0
    - Identifies if the exponent field is all 0’s
      - Special cases: Zero or Subnormal
  - s0
    - Identifies if the significand field is all 0’s

Stage 3: Exception Detection (PLA)

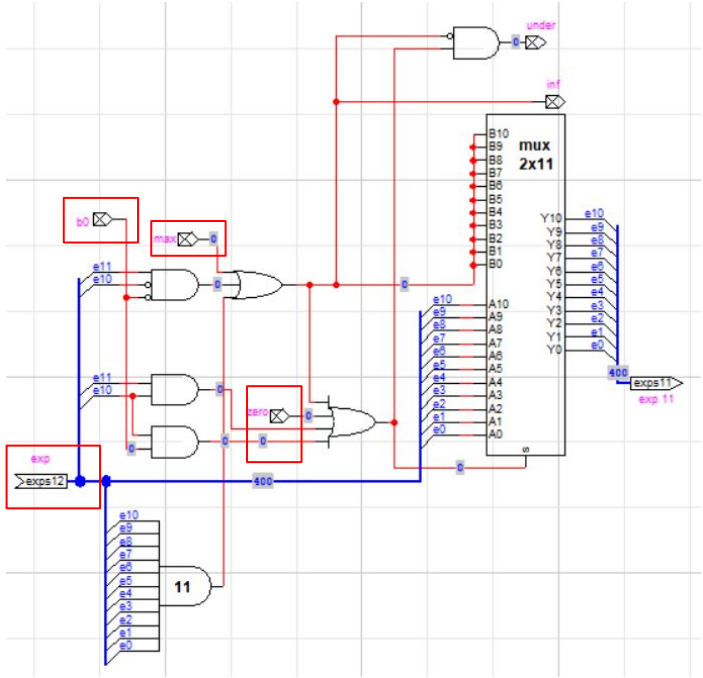
- The PLA is configure to take the inputs from the corner cases and flag conditions thats have been detected
  - NaN
    - Not a Number detected
      - X100XXXX
      - XXXXX100
      - X101X101
      - XX11XX11
  - div0
    - Division-by-zero detected (denominator)
      - X00XXX11
      - X010XX11
      - X101XX11
      - XX11XX11
  - invalid
    - Invalid operation (example 0/0, inf/inf)
      - X00XX010
      - XX10X010
  - zero
    - zero detected (numerator)
      - X011X1X1
      - X011X00X
  - max
    - maximum finite value detected
      - X1X1XXXX
      - X1X0XXXX
      - XXXXX100
      - X00XX011
      - X01XX011
  - sign
    - final result sign bit detected
      - 0XXX1XXX
      - 1XXX0XXX
      - XX11XX11

Stage 4: Clip Logic



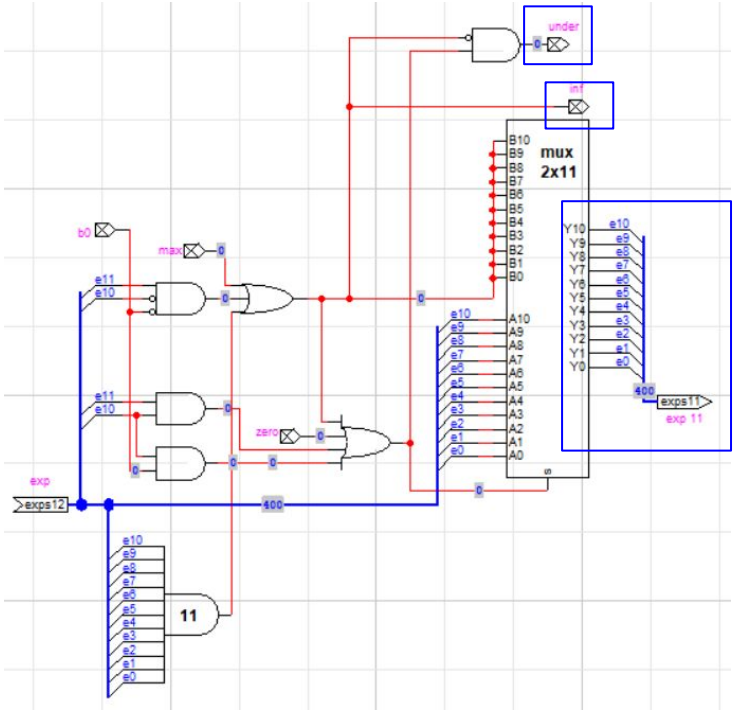
Stage 4: Clip Logic

- The clip logic is needed to make sure the exponent is valid after subtraction and biasing. Without the clip logic is could output the wrong exponent when the results is too small (underflow) or too big (infinity)
  - Inputs
    - bo
      - from subtractor (Stage 2a)
    - Max
      - PLA (Stage 3)
    - Zero
      - PLA (Stage 4)
    - Exp
      - From adder (Stage 2b)

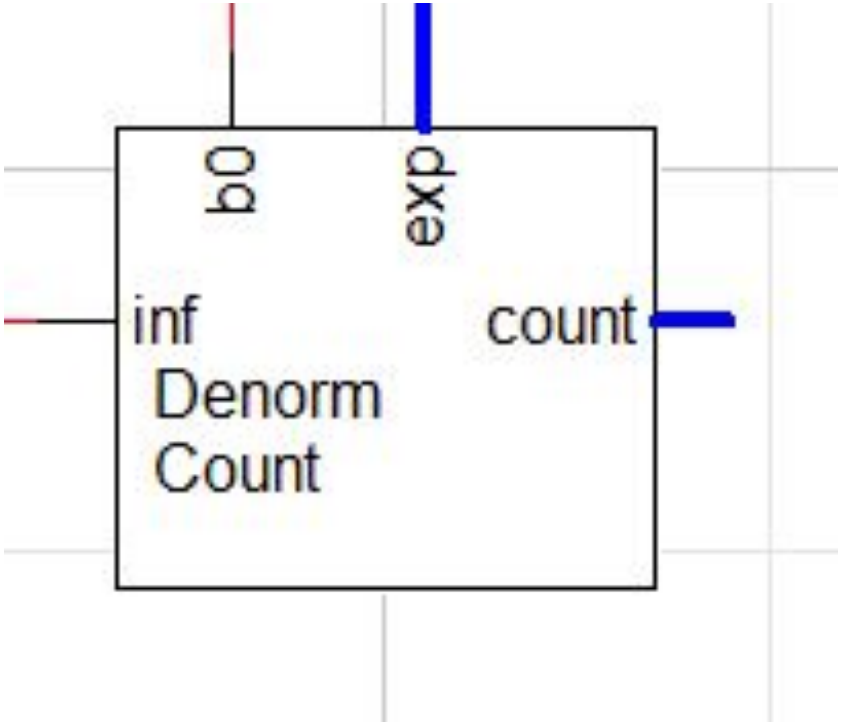


Stage 4: Clip Logic

- The output of the Clip is the adjusted exponent, the underflow and infinity flags, and then the sign bit is added back to prepare for the next stage.
  - Underflow
    - Detected when the exponent is too small to represent normally
  - Infinity
    - Detected when the exponent is too large to represent normally
  - Exponent
    - After the Clip Logic, the sign bit is added to the exponent output to complete the floating-point result
  - The Clip ensures the final exponent is valid by limiting (or “clipping”) its value to stay within the representable range.

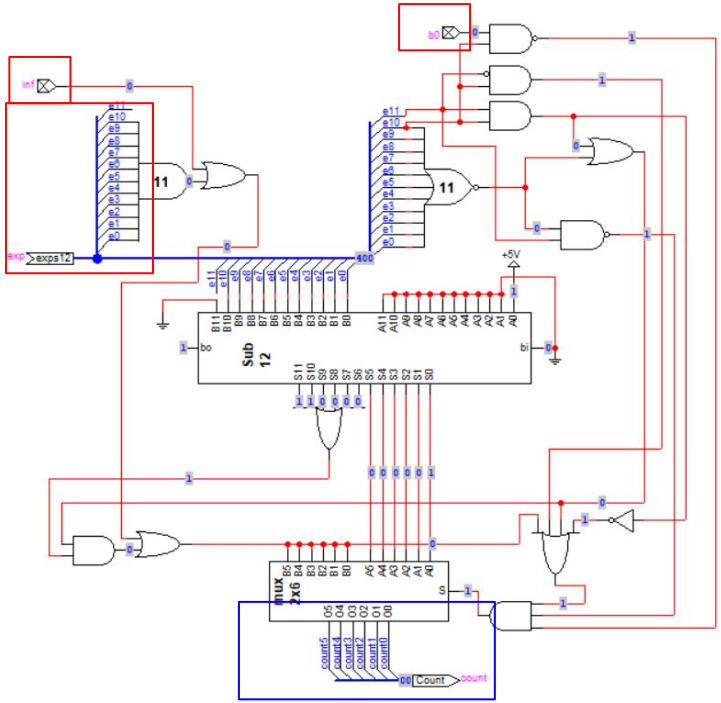


Stage 5: Denormalization

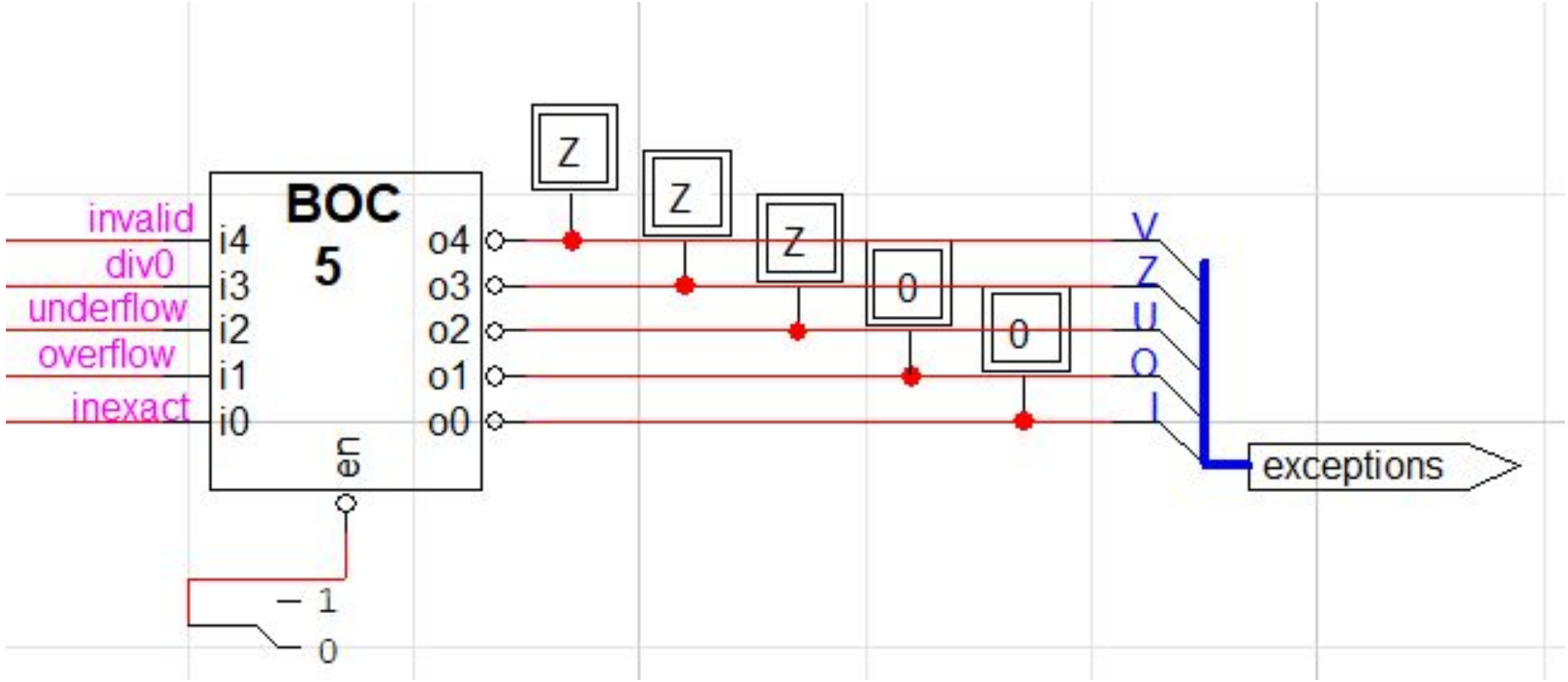


Stage 5: Denormalization

- Denormalization lets the results that are too small still be represented by forcing the exponent to zero. It prevents losing tiny numbers completely
- Inputs
  - inf
    - Informs us the result should be maximum (Stage 4 - Clip)
      - Denormalization will not happen
  - bo
    - Informs us if underflow happened (Stage 2a - Exponent Subtraction)
      - Denormalization will happen
  - exp
    - From the adder (Stage 2b - Bias Addition)
- Output
  - Count
    - Represents the adjusted exponent value after clipping (Stage 4) and denormalization
    - The output count is 6 bits because our divider only needs to represent smaller exponents, and the extra bits from math are no longer needed
    - The purpose of the Count is to represent how much the mantissa needs to be right-shifted during denormalization when the exponent is too small to be represented normally.

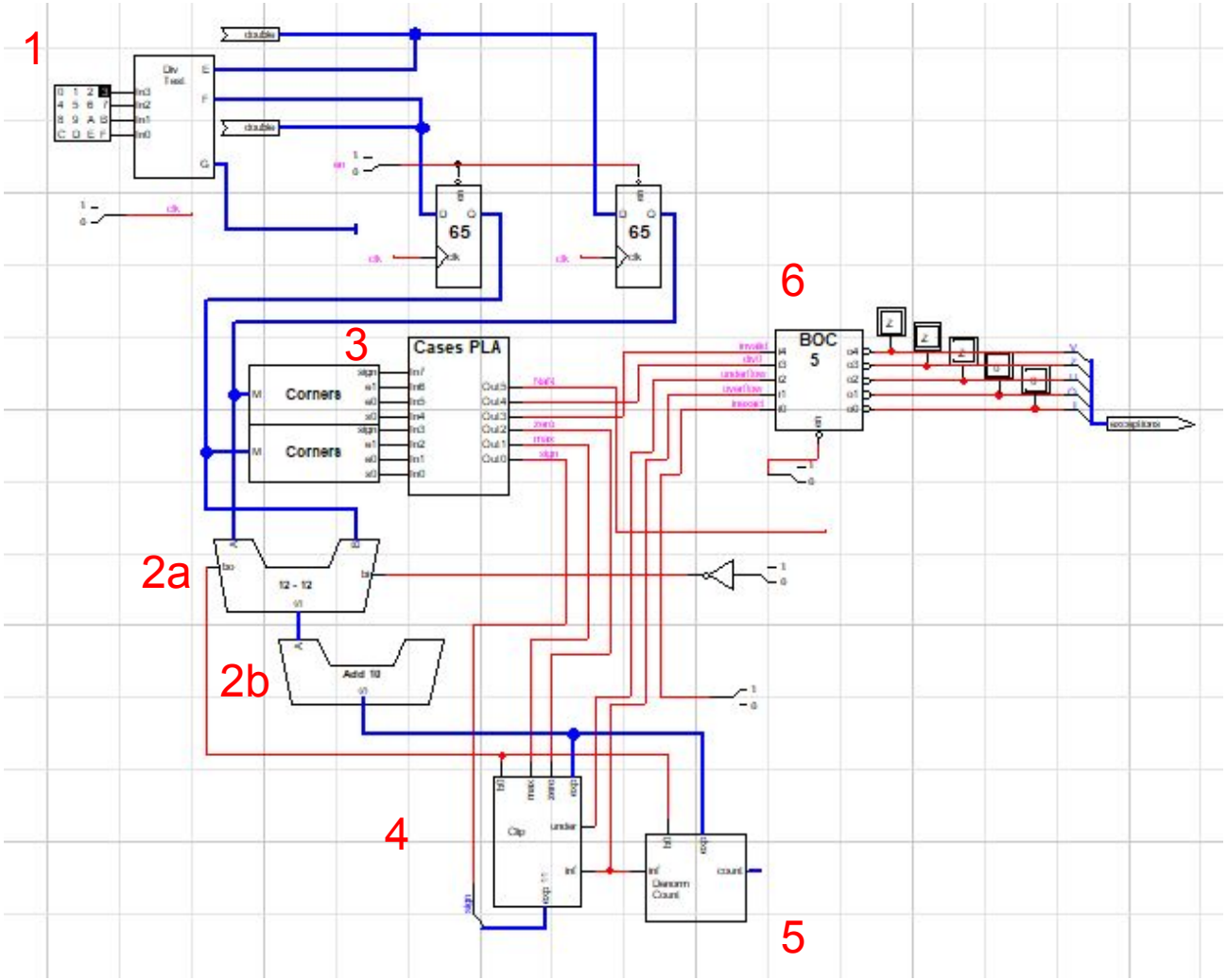


Stage 6: Exceptions



Stage 6: Exceptions

- The flags are collected into an open collected buffer to hold and export the final exception signals for the next stage
  - Inputs/Outputs
    - Invalid (V)
      - Detected from PLA (Stage 3)
    - Div0 (Z)
      - Detected from PLA (Stage 3)
    - Overflow (O)
      - Detected from inf flag from Clip (Stage 4)
    - Underflow (U)
      - Detected from Clip (Stage 4)
    - Inexact (I)
      - Detected from Round PLA and Denormalization from Mantissa
  - If an exception is flagged, it will be collected as a zero (0). If no exception is flagged and the result is not zero, it will remain as Z.


1
Div Test
double
double
65
65
clk
3
Cases PLA
Corners
Corners
6
BOC 5
2a
12 - 12
2b
Add 10
4
Clip
5
Denorm Count